디자인 패턴

2026. 3. 24 w2
미림 마이스터고

남궁성
seong.namkung@gmail.com

3. Prototype – 미리 생성한 인스턴스 복제. clone()

객체 얻는 방법

1. 항상 생성 – new

2. 생성 or 재사용 – factory method

3. 복제 - clone()

3. Prototype – 미리 생성한 인스턴스 복제. clone()

```java
class Engine{}  2 usages
class Door{}  2 usages
class Car implements Cloneable {  10 usages
    private static Car instance = new Car();  1 usage
    private Engine e;  1 usage
    private Door d;  1 usage
    Car() {  2 usages
        this.e = new Engine();
        this.d = new Door();
    }

    static Car getInstance() {  2 usages
        Car car = null;
        try {
            car = (Car)instance.clone(); // 기존의 객체를 복제
        } catch(Exception e) {
            car = new Car(); // 객체를 새로 생성
        }
        return car;
    }
}
```

```java
public class PrototypeTest {
    public static void main(String[] args) {
        Car car  = Car.getInstance();
        Car car2 = Car.getInstance();
        System.out.println("car = " + car);
        System.out.println("car2 = " + car2);
    }
}
```

```
car = Car@72ea2f77
car2 = Car@33c7353a
```

4. Builder – 생성자의 초기화 단점 해결

```java
class Student {  2 usages
    private String name;  2 usages
    private int age;  2 usages
    private int hak;  2 usages
    private int ban;  2 usages

    // 단점 2: 생성자를 많이 만들어야 한다.
    Student() { this( name: "Unknown",  age: 17,  hak: 1,  ban: 1); }  no usages
    Student(String name) { this(name,  age: 17,  hak: 1,  ban: 1); }  no usages
    Student(String name, int age) { this(name, age,  hak: 1,  ban: 1); }  no usages

    public Student(String name, int age, int hak, int ban) {  4 usages
        this.name = name;
        this.age = age;
        this.hak = hak;
        this.ban = ban;
    }

    public String toString() {
        return "Student{" + "name='" + name + '\'' + ", age=" + age +
                ", hak=" + hak + ", ban=" + ban + '}';
    }
}
```

```java
public class BuilderTest {
    public static void main(String[] args) {
        // 단점 1: 매개변수가 많아지면, 순서가 바뀔 수 있다.
        Student s = new Student( name: "kim",  age: 1,  hak: 1,  ban: 1);
        System.out.println("s = " + s);
    }
}
```

4. Builder – 생성자의 초기화 단점 해결.

```java
class Student2 {  5 usages
    private String name;  2 usages
    private int age;  2 usages
    private int hak;  2 usages
    private int ban;  2 usages

    static class Builder {  5 usages
        private Student2 student;  6 usages

        Builder() {  1 usage
            this.student = new Student2();
        }

        public Builder setName(String name) { student.name = name; return this;}
        public Builder setAge(int age) { student.age = age; return this;}  1 usage
        public Builder setHak(int hak) { student.hak = hak; return this;}  1 usage
        public Builder setBan(int ban) { student.ban = ban; return this;}  1 usage
        public Student2 build() { return student; }  1 usage
    }

    public String toString() {
        return "Student{" + "name='" + name + '\'' + ", age=" + age +
                ", hak=" + hak + ", ban=" + ban + '}';
    }
}
```

```java
public class BuilderTest2 {
    public static void main(String[] args) {
        Student2 student = new Student2.Builder()
                .setName("kim").setAge(17).setHak(1).setBan(1)
                .build();
        System.out.println("student = " + student);
    }
}
```

4. Builder – 생성자의 초기화 단점 해결.(최종)

```java
class Student3 {  4 usages
    private String name;  2 usages
    private int age;  2 usages
    private int hak;  2 usages
    private int ban;  2 usages

    Student3(Builder builder){  1 usage
        this.name = builder.name;
        this.age = builder.age;
        this.hak = builder.hak;
        this.ban = builder.ban;
    }

    static class Builder {  4 usages
        // 필수 입력은 초기값x
        private String name;  2 usages
        private int age;  2 usages

        // 선택 입력은 초기값
        private int hak = 1;  2 usages
        private int ban = 1;  2 usages

        // 필수 값은 생성자로 받는다.
        Builder(String name, int age) {  1 usage
            this.name = name;
            this.age = age;
        }
        public Builder setHak(int hak) { this.hak = hak; return this;}  1 usage
        public Builder setBan(int ban) { this.ban = ban; return this;}  1 usage
        public Student3 build() { return new Student3( builder: this); }  1 usage
    }
```

```java
public class BuilderTest3 {
    public static void main(String[] args) {
        Student3 student = new Student3.Builder( name: "kim",  age: 17) // 필수값
                .setHak(2).setBan(4) // 선택값
                .build();
        System.out.println("student = " + student);
    }
}
```